Architettura e tech stack

Documentazione tecnica del progetto Footballstats / MyCombo srl. Edizione del 11 ottobre 2026, basata sul codice locale ispezionato.

Configurazione principale: applicazione web Python ASGI con Starlette, database PostgreSQL su Supabase, frontend HTML/CSS/JavaScript e deployment Vercel. Il sistema legge un archivio persistente e produce statistiche descrittive; non contiene ancora un modello AI di forecasting.

Perimetro e attendibilità

Il documento distingue quattro livelli: codice applicativo, dipendenze dichiarate, versioni risolte nel lockfile e pacchetti installati nel virtualenv. Non è stato interrogato il runtime Vercel per certificare le versioni effettivamente caricate, né il server PostgreSQL per determinarne la major version. Le funzionalità descritte corrispondono al checkout locale: la loro presenza online dipende dall'ultimo deploy.

Le informazioni sulle fonti e sulla copertura riflettono l'archivio esaminato durante lo sviluppo. Non costituiscono una garanzia di disponibilità futura dei provider. Nel documento non sono riportati token, password o stringhe di connessione.

Sezione	Contenuto
02	Topologia e percorsi di esecuzione
03-04	Runtime Python, ASGI, API e moduli applicativi
05-06	Schema PostgreSQL, importazioni, sicurezza e cache
07-08	Acquisizione e semantica delle statistiche
09-10	Frontend, logo/animazioni e generazione Excel
11-12	Test, deploy, MCP e dipendenze
13	Limiti tecnici e riferimenti al codice

Lettura architetturale

La separazione fondamentale è fra acquisizione dei dati e consultazione. I visitatori non avviano download dai feed. Gli script di acquisizione scrivono file locali; un importatore aggiorna PostgreSQL; il backend serve report dal database, usando cache in memoria.

Topologia del sistema

Due percorsi separati: aggiornamento dell’archivio e consultazione da parte degli utenti.

Fase	Componente	Risultato
Acquisizione	CLI Python + HTTP	JSON e CSV locali con provenienza
Normalizzazione	Moduli stats e audit	Osservazioni coerenti, valori mancanti espliciti
Persistenza	Importatore Psycopg	Tabelle relazionali + payload JSONB
Consultazione	Starlette su Vercel	Report JSON da snapshot PostgreSQL
Presentazione	Browser + JavaScript	Filtri, tabelle, grafica e XLSX

Percorso di una richiesta utente

Il browser richiede una pagina HTML e i suoi asset. La pagina Esplora esegue un GET a /api/providers/report con parametri di query. Il backend valida i parametri, recupera lo snapshot dalla cache o da PostgreSQL e cerca il report nella cache delle elaborazioni. Su un miss, applica filtri e aggregazioni in Python, serializza il JSON e genera un ETag. Il browser rende la tabella e mostra 30 righe per pagina.

Percorso di un aggiornamento

L’acquisizione è un processo CLI distinto. La cache dei download evita richieste ripetute; i dati salvati vengono normalizzati dall’importatore. Una transazione PostgreSQL aggiorna le osservazioni e incrementa dataset_state.revision. Al successivo refresh dello snapshot, ogni istanza web vede la nuova revisione e invalida i propri report.

Confini di responsabilità

Vercel ospita applicazione e asset. **Supabase** ospita il database persistente. **Il filesystem locale** conserva le risposte originali per le importazioni manuali. Gli asset web non dipendono dai CSV locali in produzione: data/ è esclusa dal bundle Vercel.

Le cache non sostituiscono l’archivio. La perdita della memoria di un processo Vercel elimina solo copie temporanee; i dati PostgreSQL rimangono. La perdita dei file locali, invece, rimuoverebbe le risposte originali non altrimenti salvate: il database conserva osservazioni normalizzate, non necessariamente ogni campo dei payload upstream.

Python, ASGI e dipendenze

Runtime richiesto: Python >=3.13,<3.14. Interprete locale rilevato: Python 3.13.7.

Libreria	Vincolo	uv.lock	Locale
Starlette	>=1.6,<2	1.7.0	1.6.0
Uvicorn	>=0.52,<1	0.54.0	0.52.4
Psycopg [binary]	>=3.2,<4	3.3.6	3.3.6
psycopg-binary	Extra del driver	3.3.6	3.3.6

Starlette: interfaccia HTTP

Starlette è il framework ASGI. ASGI definisce il contratto asincrono fra server e applicazione mediante scope e canali receive/send. Nel progetto gestisce routing, richieste, risposte e file statici. Non utilizziamo FastAPI: non ci sono validazione Pydantic dei report, dependency injection di FastAPI o schema OpenAPI generato automaticamente.

Uvicorn: esecuzione locale

Lo script webapp.py può avviare Uvicorn con host e porta configurabili. Uvicorn riceve HTTP e invoca l'app ASGI. In produzione l'entrypoint dichiarato a Vercel è src.webapp:app; il server avviato dalla funzione main() è il percorso locale, non un processo persistente che configuriamo manualmente su Vercel.

Sincronia e thread pool

Le route sono async, ma Psycopg viene usato in modalità sincrona, così come le aggregazioni e la generazione XLSX. run_in_threadpool sposta queste operazioni fuori dall'event loop. Questo impedisce che una singola operazione sincrona occupi direttamente il loop, ma non rende il calcolo vettoriale o distribuito. I lock possono inoltre serializzare le elaborazioni di una stessa istanza.

Driver e riproducibilità

Psycopg 3 traduce parametri e tipi Python nel protocollo PostgreSQL; l'extra binary fornisce l'implementazione binaria pronta all'uso. Il codice usa SQL esplicito, senza ORM. Il lockfile fissa una risoluzione, mentre i vincoli nel manifest ammettono intervalli: l'ambiente locale oggi non coincide con il lockfile per Starlette e Uvicorn. Occorre allinearli prima di considerare identici ambiente testato e ambiente di build.

API e moduli applicativi

Il backend è un’applicazione compatta, con logica di dominio separata dal routing.

Metodo / percorso	Responsabilità
GET /	FileResponse della pagina aziendale
GET /providers	FileResponse dell’esploratore
GET /documentation	Documentazione e controlli sui dati
GET /api/providers/report	Report JSON filtrato; ETag e cache
POST /api/export.xlsx	Tabella JSON convertita in XLSX
GET /static/*	StaticFiles per HTML accessori, CSS, JS e SVG

webapp.py e validazione

Route e Mount collegano gli handler all’app. FileResponse, JSONResponse e Response coprono HTML, JSON, byte binari e 304. I filtri includono kind, metric, league, season, team, search, min_minutes, last_n e sort. Esistono limiti sulla lunghezza della query e delle stringhe, conversioni numeriche e whitelist per tipo, metrica e ordinamento. last_n è limitato a 100. Gli errori di parametri restituiscono 400; un database non disponibile restituisce 503 con Retry-After.

provider_stats.py

È il motore dei report: riceve osservazioni locali o uno snapshot cloud, riconcilia fonti sovrapposte, raggruppa giocatore/squadra per fonte, campionato e stagione, calcola totali e copertura, ordina e restituisce metadati. La ricerca viene applicata sui nomi; il periodo ultime N seleziona prima le presenze e poi verifica la disponibilità della statistica.

Moduli di supporto

cloud_store.py gestisce connessione e snapshot. **report_cache.py** conserva JSON già serializzati. **csv_stats.py** e **understat_stats.py** normalizzano le fonti. **score_audit.py** controlla punteggi e periodi. **lineup_stats.py** costruisce la base titolare. **excel_export.py** produce il workbook. La separazione permette di testare calcoli e parsing senza effettuare richieste reali ai provider.

L’API è di consultazione: non espone query SQL arbitrarie o comandi di acquisizione. L’export POST genera una risposta e non modifica il database.

PostgreSQL e JSONB

Supabase fornisce il PostgreSQL persistente. L’app si collega direttamente con Psycopg attraverso il pooler.

Tabella nello schema football	Colonne / ruolo
teams	provider, id, name; chiave composta (provider,id)
players	provider, id, name; identità separata per fonte
fixtures	provider, id, match_date, league, season
team_match_stats	provider, fixture_id, team_id, payload JSONB
player_match_stats	provider, fixture_id, player_id, team_id, payload JSONB
dataset_state	id, revision, metadata JSONB, updated_at

Schema fisso, contenuto estensibile

Le colonne PostgreSQL rimangono fisse. payload contiene un oggetto JSONB con nome, squadra, data, minuti, titolarità e un oggetto stats. Campi come shots-total o fouls-drawn sono chiavi del JSON, non colonne relazionali aggiunte dinamicamente. La base stagionale viene invece calcolata a runtime. L’aggiunta di una nuova colonna SQL richiederebbe una migrazione esplicita.

Integrità e identità

Primary key e foreign key collegano osservazioni, partite ed entità della medesima fonte. Il check su payload impone un oggetto JSON; non valida ogni statistica interna, responsabilità che rimane al codice. L’indice su fixtures(league,season,match_date) supporta accessi per calendario, anche se il lettore attuale non applica questi filtri nel database.

La chiave (provider,id) evita collisioni fra ID di provider diversi, ma non costituisce un’identità universale del calciatore. I conteggi distinti sono basati su questo namespace. Lo stesso soggetto in più fonti potrebbe richiedere una tabella di riconciliazione aggiuntiva; i nomi da soli non sono un identificatore affidabile.

Accesso e permessi

Lo schema football è privato, con RLS e policy dedicate. football_reader ha SELECT; football_importer ha i permessi necessari alle importazioni. Il ruolo amministrativo è separato. Non usiamo supabase-py, PostgREST, Auth, Realtime o Storage nel percorso applicativo. Il frontend non riceve credenziali PostgreSQL.

Transazioni e cache

Consistenza del database, isolamento delle credenziali e riutilizzo dei risultati sono meccanismi diversi.

Lettura coerente

cloud_store apre una connessione con connect_timeout=10 e prepare_threshold=None. Per host remoti impone almeno sslmode=require, salvo modalità TLS già più restrittive. Questo abilita cifratura, ma require non equivale alla verifica hostname di verify-full. Non è configurato un pool Psycopg interno; il pooling è fornito dall'endpoint Supabase.

La lettura usa REPEATABLE READ READ ONLY e statement_timeout di 15 secondi. Stato, metadati e osservazioni vengono letti nella stessa transazione: un import concorrente non produce un mix di revisioni. Vengono caricati tutti i payload di team e giocatori; filtri e aggregazioni avvengono dopo, in memoria.

Importazione atomica

supabase_data.py valida l'archivio locale, avvia una transazione e blocca la riga dataset_state con FOR UPDATE. Esegue upsert delle entità e partite, sostituisce le osservazioni solo per le fixture importate e incrementa revision. Le altre partite vengono conservate. Il lettore vede lo stato precedente o quello successivo al commit. Il lock coordina le importazioni, non i download dai provider.

Cache	Politica e limite
Snapshot DB	TTL 60 s; Lock per processo; backoff errori 15 s
Report JSON	TTL 60 s; LRU; 32 entry; 8 MiB di corpi JSON
Invalidazione	Revisione cambiata: svuotamento cache report
HTTP	ETag SHA-256; Cache-Control: no-cache; 304 condizionale
Download	File locali riutilizzati dai comandi di acquisizione

Le cache in memoria non sono distribuite. Ogni istanza Vercel può aprire una propria lettura PostgreSQL. L'ETag riduce il trasferimento quando il contenuto coincide; non elimina necessariamente il lavoro sul server, che deve conoscere il report corrente. Il limite di 8 MiB riguarda la cache report, non la dimensione dello snapshot né tutta la memoria del processo.

Acquisizione dei feed

HTTP dalla standard library, cache su disco e importazione manuale. Nessun downloader viene avviato dalla navigazione.

Fonte	Implementazione	Contributo attuale
Sportmonks	API ufficiale, token in header	Superliga e Premiership scozzese
API-Football	API ufficiale, x-apisports-key	Connettore presente; ultimo controllo: account sospeso
Understat	Endpoint JSON del sito, senza chiave	Giocatori e squadre dei cinque grandi campionati
football-data.co.uk	Download CSV	Statistiche squadra e punteggi HT espliciti

fetch_provider.py

Usa urllib.request per HTTP e urllib.parse per i parametri. Legge le chiavi dall'ambiente o dal file locale tramite un parser dedicato. Limita le richieste per esecuzione, usa una cache indicizzata dalla richiesta e salva JSON con provenienza e timestamp. Gestisce errori HTTP e payload non validi; evita di salvare token e URL di paginazione contenenti credenziali. La disponibilità di un endpoint non implica che il piano includa tutti i campionati.

fetch_understat.py

Richiede getLeagueData/{league}/{season} e getMatchData/{id}. Non fa parsing HTML né automazione browser. È un'integrazione non ufficiale degli endpoint del sito, non un contratto API garantito. Gestisce JSON e risposte gzip, valida chiavi strutturali, scrive tramite file temporaneo e replace. Riusa la cache salvo flag di refresh, procede sequenzialmente e applica una pausa fra richieste e un budget massimo.

Persistenza delle risposte e limiti

La cache dei file Understat non ha un TTL automatico: evita sprechi, ma richiede una scelta esplicita su quando ricontrollare risultati corretti. Non sono ancora implementati budget giornaliero distribuito, lock globale delle acquisizioni, scheduler o gestione dedicata di Retry-After. In caso di indisponibilità della fonte, l'archivio PostgreSQL resta consultabile.

urllib, json, csv e gzip sono moduli della standard library: non richiedono Requests, BeautifulSoup o Scrapy. Non sono stati implementati proxy rotanti o aggiramenti dei blocchi.

Qualità e semantica dei dati

Una metrica è utilizzabile solo se sono chiari campione, definizione, provenienza e valori mancanti.

Normalizzazione e riconciliazione

Understat e CSV possono descrivere la stessa partita. Il merge richiede campionato, stagione, data, entrambe le squadre normalizzate tramite alias espliciti e compatibilità del punteggio finale. Nessun fuzzy matching è usato. Le osservazioni delle fonti restano separate nel database; il report le riconcilia per evitare doppie partite. Un conflitto FT esclude la fixture Understat dal report e genera un avviso.

Totali, copertura e valori per 90 minuti

Un valore mancante resta null, distinto da zero. I totali sommano solo le osservazioni note. La copertura indica quante presenze contengono il dato rispetto al campione. Per 90 minuti = totale noto × 90 / minuti delle presenze con quel dato. La media per partita divide per le partite con statistica disponibile. Se una fonte omette gli zeri, queste medie possono risultare selettive: la copertura è indispensabile.

Tiri, falli e primo tempo

Tiri totali e tiri in porta sono distinti. Per gli eventi Understat, i tiri in porta sono Goal + SavedShot solo quando numero e classificazione degli eventi sono coerenti con il totale dichiarato. I falli subiti usano fouls-drawn e non vengono derivati dai falli avversari. HT usa punteggi espliciti Sportmonks 1ST_HALF o CSV HTHG/HTAG: non viene stimato dal minuto <=45.

score_audit verifica gli eventi per periodo, comprende il recupero, ignora eventi annullati e controlla la progressione del punteggio. È una verifica interna al feed, non una conferma indipendente. Le incoerenze nei gol individuali e autogol possono escludere i giocatori della partita preservando i dati squadra validi.

Formazione abituale

lineup_stats considera le formazioni complete di 11 ID distinti, raggruppate per fonte, campionato, stagione e squadra. Ordina per partenze da titolare, poi minuti e ID; segnala il pari merito al taglio. La base è una graduatoria di frequenza, non un modulo tattico. Il confronto riguarda l’ultima partita inclusa nella base. Non è una previsione out-of-sample: prossime formazioni, assenze e infortuni non sono acquisiti.

Frontend e interazione

Pagine statiche, JavaScript vanilla e due librerie di animazione servite localmente.

Componente	Responsabilità
index.html + landing.css/js	Presentazione aziendale e percorso narrativo
providers.html/js + explorer.css	Filtri, viste squadra/giocatore, tabella e paginazione
documentation.html/js	Copertura, spiegazioni e audit del primo tempo
brand.css/js + owl-logo.svg	Marchio, riflesso luminoso e gufo cursore
excel.js	Raccolta tabella e download del workbook

GSAP e ScrollTrigger 3.13.0

GSAP orchestra timeline e interpolazioni; ScrollTrigger collega animazioni e avanzamento allo scroll. Il codice usa gsap.matchMedia per attivare animazioni e percorso orizzontale fissato solo nei contesti appropriati. Le distribuzioni JS sono vendorizzate in static/vendor: nessuna dipendenza dal CDN durante la visita. Le pagine restano leggibili se le animazioni non partono.

Dati e stato UI

fetch carica il report; AbortController e un contatore di generazione evitano che risposte vecchie sostituiscano filtri nuovi. URLSearchParams e history.replaceState mantengono i filtri nell’URL. La ricerca è ritardata con debounce. La tabella viene costruita con API DOM e textContent; i risultati sono paginati in blocchi da 30 nel browser, non nel database.

Logo e cursore

Il logo è SVG. L’effetto sul nome è un gradiente CSS animato; il gufo segue il puntatore tramite requestAnimationFrame e interpolazione dipendente dal tempo. Quando raggiunge la posizione obiettivo il ciclo si ferma. pointer-events:none evita di intercettare i clic; blur, uscita del puntatore e cambio di visibilità nascondono il follower. prefers-reduced-motion e capacità touch disabilitano il movimento.

Non sono presenti framework component-based, virtual DOM, TypeScript, bundler, Tailwind o Bootstrap. HTML/CSS/JS sono pubblicati direttamente. Node è stato usato per controlli sintattici, non è il runtime del backend.

Excel e standard library

Un XLSX è un pacchetto ZIP contenente documenti XML OOXML: il progetto lo genera direttamente.

Flusso di esportazione

excel.js costruisce una tabella completa dei risultati già caricati, non soltanto della pagina visibile. Conserva numeri e null tramite valori associati alle celle e aggiunge colonne per note/copertura. Invia la matrice JSON a /api/export.xlsx. Il backend non rilegge provider o database: converte la matrice ricevuta in un file e risponde con il MIME XLSX e Content-Disposition attachment.

Implementazione e limiti

excel_export.py scrive workbook, relazioni, stili e worksheet usando zipfile e XML. L'output ha un foglio Dati, intestazioni evidenziate, filtro e prima riga bloccata. Le stringhe vengono serializzate come inlineStr: un nome che inizia con = non diventa una formula. I numeri restano numeri; i null non diventano zeri. Vengono controllati dimensione del body (4.000.000 byte), righe (10.000), colonne (80), celle (200.000) e lunghezza delle stringhe.

Non è una libreria OOXML generale: non gestisce formule, grafici o pivot. Per workbook più complessi occorrerebbe estendere o sostituire il writer. I test verificano struttura ZIP/XML, tipi e download; non equivalgono a una certificazione di ogni versione di Excel.

Moduli standard	Impiego nel progetto
json, csv, gzip	Parsing, serializzazione e decompressione
urllib.request/error/parse	HTTP, errori e costruzione delle URL
collections, functools	Raggruppamenti, LRU locale e dizionari ordinati
threading, time	Lock, TTL monotono e pause acquisizione
datetime, pathlib, os, sys	Date, percorsi, ambiente e avvio
hashlib, secrets	ETag/cache key e generazione credenziali
io, zipfile, xml.sax.saxutils	XLSX in memoria ed escaping XML
argparse, re, math	CLI, validazione testuale e numerica
unittest, tempfile, copy, concurrent.futures	Test, fixture isolate e verifica concorrenza

Test, deployment e MCP

La pipeline di test è distinta dalla build Vercel e dal server MCP opzionale.

Verifiche automatiche

La suite usa unittest e unittest.mock. Starlette TestClient esegue richieste contro l'app senza avviare un server pubblico; i mock evitano consumo dei feed. ThreadPoolExecutor verifica che richieste concorrenti condividano una lettura o un calcolo nella stessa istanza. Altri test coprono parsing, HT, merge, snapshot JSON, valori mancanti, XLSX e base titolare. L'ultima esecuzione riportata nello sviluppo ha superato 36 test; non è stata rieseguita per redigere questo PDF.

requirements-dev.txt dichiara httpx>=0.28,<1, mentre l'ambiente locale contiene httpx2 2.12.0 e il TestClient installato ne supporta l'import. Il manifest di sviluppo e il virtualenv devono essere riallineati per poter riprodurre con precisione le verifiche in un ambiente pulito.

Build e distribuzione

Git/GitHub conserva il codice; l'integrazione Vercel pubblica il branch configurato. pyproject.toml dichiara il progetto, il range Python e src.webapp:app; uv.lock registra la risoluzione delle dipendenze. tool.uv.package=false evita di trattare il repository come pacchetto da costruire. .vercelignore esclude segreti, dati locali, script, artefatti e componente MCP.

Il push aggiorna il codice, non PostgreSQL. Per nuovi dati occorre l'importatore; per una metrica già presente nel payload può bastare il deploy. Le modifiche strutturali SQL richiedono una migrazione. Non risultano configurati nel progetto un job pianificato di acquisizione o un workflow CI che esegua automaticamente tutta la suite.

MCP e DuckDB: percorso opzionale

requirements-mcp.txt aggiunge duckdb>=1.1 e mcp[cli]>=2,<3. Il virtualenv contiene DuckDB 1.5.5 e MCP 2.2.0. server.py usa MCPServer per il protocollo stdio; db.py applica una connessione DuckDB read_only con enable_external_access=false, oltre a controlli SQL testuali. ingest.py e schema.py appartengono a quel percorso locale.

Questi file e i database DuckDB sono esclusi dal deploy web. Pydantic, librerie CLI e altre dipendenze dell'ambiente MCP non vanno confuse con librerie direttamente usate dalla dashboard. Nessun LLM viene invocato dal report HTTP.

Inventario delle librerie

Versioni rilevate nell’ispezione. La presenza nel virtualenv non implica che il sito importi il pacchetto.

Dipendenza transitiva nel lock web	Versione	Funzione
anyio	4.15.1	Primitivi di concorrenza usati da Starlette
idna	3.20	Gestione nomi di dominio internazionalizzati
typing-extensions	4.16.0	Supporto typing compatibile
click	8.5.0	Interfaccia CLI di Uvicorn
h11	0.16.0	Protocollo HTTP/1.1 di Uvicorn
tzdata	2026.5	Database fusi orari, secondo piattaforma
psycopg-binary	3.3.6	Implementazione binaria Psycopg

Ulteriori pacchetti presenti nel virtualenv

Inventario locale aggiuntivo, raggruppato per area funzionale e senza attribuire a ciascuno una dipendenza diretta del sito. idna locale è 3.19, diverso dal 3.20 del lockfile.

Tipi e validazione: annotated-doc 0.0.5; annotated-types 0.8.0; attrs 26.1.0; pydantic 2.13.5; pydantic_core 2.46.5; typing-inspection 0.4.4.

JSON Schema: jsonschema 4.26.0; jsonschema-specifications 2025.9.1; referencing 0.37.0; rpds-py 2026.6.3.

HTTP, streaming e osservabilità: httpx2 2.12.0; httpcore2 2.12.0; sse-starlette 3.4.11; opentelemetry-api 1.44.0.

Crittografia e sistema: cryptography 50.0.1; cffi 2.1.1; pycparser 3.0; PyJWT 2.13.0; truststore 0.10.4; pywin32 312.

CLI e testo: typer 0.27.2; shellingham 1.5.4; rich 15.0.0; Pygments 2.21.0; markdown-it-py 4.2.0; mdurl 0.1.2; colorama 0.4.6.

Utility e strumenti: python-dotenv 1.2.3; python-multipart 0.0.32; pglast 8.5; pip 26.2.1; uv 0.13.0; mcp-types 2.2.0.

Il codice di acquisizione usa un parser .env proprio, non python-dotenv. La presenza di PyJWT non indica autenticazione JWT degli utenti; quella di OpenTelemetry non dimostra una pipeline di telemetria configurata. ReportLab e pypdf, usati per questo documento, provengono dal runtime di authoring e non sono dipendenze del progetto.

Limiti e riferimenti tecnici

Distinzione fra comportamento implementato e possibili evoluzioni; nessuna modifica infrastrutturale è stata eseguita per questo documento.

Punti da mantenere espliciti

Scalabilità: caricamento integrale dell'archivio, aggregazioni Python, serializzazione completa e paginazione client-side fanno crescere CPU, memoria e traffico con i dati. Un'evoluzione può spostare filtri e aggregazioni in SQL e paginare lato server, mantenendo identica la semantica dei valori mancanti.

Coordinamento: Lock e LRU sono locali al processo. Un futuro scheduler richiederà controllo condiviso di budget, esclusione dei job concorrenti e cache delle risposte accessibile fra esecuzioni. Il lock dell'importatore protegge solo il commit dell'archivio.

Semantica: la base titolare non misura forma fisica o disponibilità futura. L'identità per provider non equivale a una riconciliazione universale dei giocatori. Una copertura incompleta deve restare visibile anche negli export e in un futuro dataset di training.

Riproducibilità: riallineare versioni locali e lockfile, verificare installazione pulita, e registrare versioni del runtime effettivamente distribuito. PostgreSQL major e pacchetti attivi su Vercel non sono stati verificati in questa ispezione.

File sorgente di riferimento, relativi alla root del repository

Area	File principali
Manifesti e build	pyproject.toml; uv.lock; requirements*.txt; .python-version; .vercelignore
HTTP e cache	src/webapp.py; src/cloud_store.py; src/report_cache.py
Dominio	src/provider_stats.py; src/understat_stats.py; src/csv_stats.py; src/score_audit.py; src/lineup_stats.py
Persistenza	supabase/migrations/202610100001_football.sql; scripts/supabase_data.py
Acquisizione	scripts/fetch_provider.py; scripts/fetch_understat.py; scripts/fetch_football_data.py
Frontend e export	src/static/; src/static/vendor/README.md; src/excel_export.py
Test e MCP	scripts/test_*.py; scripts/smoke_test.py; src/server.py; src/db.py

Questa descrizione deriva dai file del progetto e dall'inventario locale, non da supposizioni sulle funzionalità dei servizi cloud. Le versioni sono una fotografia dell'ispezione, non un elenco delle release più recenti.